

OncoLake

Un data lake pour classer des protéines liées au cancer

Rapport technique

Projet final — Data Lakes & Data Integration
EFREI, 2025–2026

Florian DELSUC
Sedik BENMESSAOUD

July 12, 2026

1. Ce qu'on a voulu faire

Le sujet nous laissait libres sur le domaine, et on avait envie de partir sur quelque chose qui sorte un peu de l'ordinaire. On s'est retrouvés autour d'une idée de biologie : essayer de deviner, à partir de la seule forme 3D d'une protéine, si celle-ci est un oncogène ou un suppresseur de tumeur.

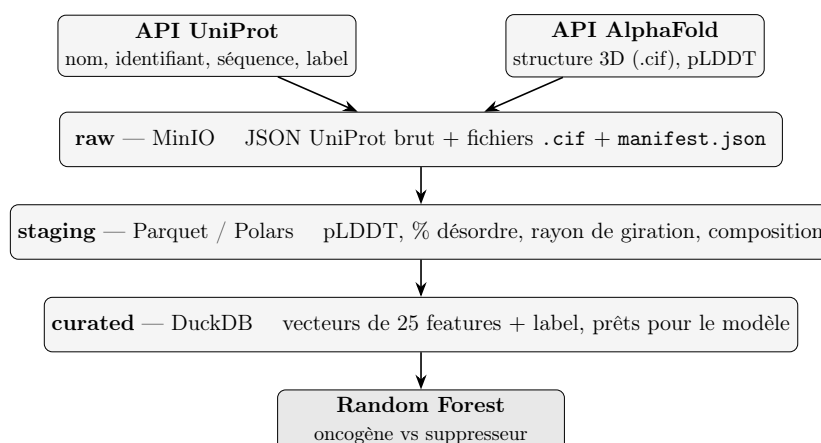
Un petit mot pour situer, parce que ce n'est pas évident au départ. Dans une cellule, deux familles de gènes se répondent : les oncogènes, qui poussent la cellule à se diviser, et les suppresseurs de tumeur, qui la freinent. Un cancer, c'est souvent l'un de ces deux mécanismes qui déraile. Comme la fonction d'une protéine dépend en grande partie de sa forme, on s'est posé la question suivante : est-ce que cette forme, à elle seule, porte la trace du rôle de la protéine ?

On avait une intuition de départ. Les protéines impliquées dans le cancer ont souvent des zones dites « désordonnées », c'est-à-dire des parties qui ne se replient pas en une structure rigide. Or AlphaFold, quand il prédit une structure, donne pour chaque acide aminé un score de confiance appelé pLDDT, et une confiance basse correspond justement, la plupart du temps, à une région désordonnée. On tenait donc une mesure du désordre « gratuite », qu'on comptait bien utiliser comme indice.

Ce rapport explique surtout **pourquoi** on a fait les choix qu'on a faits, et ce que ça a donné. Les procédures d'installation et d'utilisation sont rassemblées en section 7 (le README du dépôt les détaille davantage).

2. L'architecture : trois zones, et pourquoi

On a repris la structure raw / staging / curated vue en cours, parce qu'elle correspondait bien à notre cas. On ne voulait pas jeter la donnée brute (les téléchargements coûtent du temps et du réseau), donc on garde tout tel quel en *raw*, on transforme en *staging*, et on prépare la version finale prête pour le modèle en *curated*.



Transverses : DVC orchestre ingest→features→curate→train ; FastAPI expose /raw /staging /curated /ingest /ingest_fast

Concrètement, l'ingestion ramène **418 protéines** (231 oncogènes, 187 suppresseurs), dont **414 ont une structure** ; après retrait des cas ambigus (section suivante), il reste **404 protéines** décrites chacune par **25 features** dans la zone curated.

Le vrai point d'architecture sur lequel on a réfléchi, c'est la couture entre les zones. Nos deux sources ne parlent pas le même langage : les listes de gènes du cancer donnent des noms (TP53), alors qu'AlphaFold range ses structures par identifiant UniProt (P04637). On a résolu ça avec UniProt, qui fait le lien : une seule requête par mot-clé nous renvoie à la fois le nom, l'identifiant,

la séquence et le label. On écrit ensuite un fichier `manifest.json` dans la zone raw qui liste chaque protéine avec un drapeau indiquant si sa structure a bien été récupérée. Ce manifeste sert de table d'index : c'est lui que l'étape suivante lit pour savoir quoi traiter et quoi laisser de côté. Sans lui, les zones seraient des dossiers de fichiers sans lien entre eux.

Autre chose qu'on a appris à nos dépens : au début, on avait deux fichiers de configuration et deux modules de stockage qui faisaient à peu près la même chose, l'un masquant l'autre. Ça a fini par casser une des étapes sans qu'on comprenne pourquoi. On a tout ramené à une seule source de configuration et un seul module d'accès au stockage, et le problème a disparu. La leçon qu'on retient : mieux vaut un seul endroit qui fait autorité que deux qui se contredisent.

3. Les choix techniques

Zone raw : MinIO. Le sujet imposait S3 ou Elasticsearch pour la zone brute. On a pris MinIO parce qu'il parle le protocole S3 (donc notre code tournerait tel quel sur un vrai S3 d'Amazon en changeant juste l'adresse), mais tourne en local dans Docker, avec une vraie console web pour voir ce qu'on stocke. On avait regardé LocalStack au début, mais MinIO était plus simple à mettre en route et plus adapté à du stockage d'objets.

Lecture des structures : gemmi. Les fichiers d'AlphaFold sont au format mmCIF. On a d'abord pensé à Biopython, qu'on connaissait déjà, mais gemmi lit ce format plus vite et surtout donne directement accès au champ où AlphaFold cache le score pLDDT (le B-factor). Ça nous évitait de bricoler.

Tables de features : Polars plutôt que pandas. On voulait quelque chose de rapide avec une écriture Parquet directe, et Polars fait le travail sans dépendance lourde.

Zone curated : DuckDB. On avait besoin de requêter nos features en SQL sans monter une vraie base de données serveur. DuckDB tourne dans le processus Python et sait lire le Parquet directement depuis MinIO, ce qui collait parfaitement.

Pipeline : DVC et pas Airflow. C'est le choix qu'on a le plus discuté. Airflow est fait pour planifier des tâches qui tournent en boucle, avec un serveur, un ordonnanceur, des tableaux de bord. Nous, on n'avait pas besoin de « relancer l'ingestion toutes les nuits ». On avait besoin de pouvoir **reconstruire tout le pipeline à l'identique** et de ne refaire que ce qui a changé. C'est exactement ce que fait DVC, en local, sans serveur, et en versionnant les paramètres. Airflow aurait été un marteau pour une punaise.

Modèle : Random Forest. On aurait pu tenter du deep learning, mais avec quelques centaines d'exemples ça n'avait pas de sens, et surtout on voulait un modèle qui nous dise *quelles features comptent*, pour pouvoir tester notre hypothèse sur le désordre. Le Random Forest donne cette information directement.

API : FastAPI. Le sujet demandait une API Gateway exposant chaque couche. FastAPI valide automatiquement le format des requêtes à partir de nos types Python et génère une documentation interactive (`/docs`) sans qu'on ait une ligne à écrire, ce qui rendait la démonstration des endpoints immédiate.

Validation : Pydantic. Plutôt que de vérifier à la main la forme des données à chaque frontière de zone, on a écrit nos schémas une seule fois (`schemas.py`). Une protéine mal formée est rejetée à l'entrée, avec un message clair, au lieu de propager une erreur obscure trois étapes plus loin.

Conteneurisation : Docker. MinIO tourne dans un conteneur, et les buckets sont créés automatiquement au démarrage. L'intérêt est la reproductibilité : n'importe qui clone le dépôt, lance une commande, et obtient exactement le même environnement de stockage, sans rien installer sur sa machine.

4. Gérer les cas qui coïncident

Une bonne partie du travail a consisté à traiter proprement les cas particuliers, parce que la donnée réelle n'est jamais aussi propre qu'on l'espère.

Quatre protéines (KMT2A, APC, ATM, BRCA2) n'ont pas de structure dans AlphaFold. Après vérification, ce sont des protéines géantes, au-delà de ce qu'AlphaFold modélise en une seule chaîne. Ce n'est donc pas un bug de notre côté : on les marque dans le manifeste et on les écarte au moment de construire les features, sans faire planter le reste.

Cas plus intéressant : environ cinq protéines portent *les deux* labels à la fois, oncogène et suppresseur, parce qu'elles jouent les deux rôles selon le contexte. Elles cassaient l'unicité de l'identifiant qu'on attendait. Plutôt que de choisir à leur place, on a rendu la décision configurable (les écarter, forcer un label, ou en faire une troisième classe), avec « les écarter » par défaut, puisque des étiquettes contradictoires n'ont pas de sens pour une classification binaire.

On a aussi fait attention aux erreurs réseau : sur plusieurs centaines de téléchargements à la suite, une coupure ne doit pas faire tout échouer. Une structure absente renvoie simplement une valeur vide qui est gérée en amont.

5. Les résultats, et notre lecture

On a entraîné un Random Forest (300 arbres) sur 404 protéines (225 oncogènes, 179 suppresseurs), avec un découpage stratifié et une validation croisée à 5 plis pour ne pas dépendre d'un seul tirage.

Exactitude sur le jeu de test	0,50
Exactitude en validation croisée	0,52 (écart-type 0,04)
Baseline (toujours la classe majoritaire)	0,56

Il faut être honnête : le modèle **ne bat pas** le fait de répondre bêtement « oncogène » à chaque fois. On a regardé l'importance des features, en s'attendant à voir le désordre ressortir, et en fait rien ne se détache : toutes les features ont à peu près le même poids, y compris celles qu'on croyait décisives. La matrice de confusion confirme que le modèle reconnaît mal les suppresseurs et se rabat sur la classe la plus fréquente.

On a d'abord été déçus, puis on s'est rendu compte que c'est un résultat en soi, et pas un échec. Oncogènes et suppresseurs sont *tous les deux* des protéines du cancer : elles se ressemblent structurellement. Ce qui les distingue se joue sans doute à une échelle très fine, celle d'une mutation ou d'un partenaire d'interaction, que la forme d'ensemble ne capte pas. Notre pipeline a le mérite de rendre ce constat mesurable et explicite, au lieu de le cacher. Si on avait obtenu un très bon score, on se serait d'ailleurs méfiés : le risque classique aurait été que le modèle apprenne surtout la taille des protéines.

6. Le niveau avancé : /ingest et /ingest_fast

On expose l'ingestion via l'API. L'endpoint `/ingest` reçoit un lot de séquences, récupère leur structure et calcule les features, le tout chronométré. C'est la version « naïve », séquentielle, qui sert de point de comparaison.

L'endpoint `/ingest_fast` doit faire au moins 30 % plus vite. Le sujet suggérait plusieurs pistes (Numba, NumPy, parallélisme, asynchronicité, cache), mais elles ne se valent pas ici : encore fallait-il identifier *où* le temps passe réellement.

Or notre ingestion est **limitée par les entrées/sorties**, pas par le calcul. Sur chaque protéine, l'essentiel du temps est de l'attente réseau (interroger AlphaFold, puis télécharger le fichier),

tandis que l'extraction des features ne représente qu'une fraction minime. Nous avons donc écarté **Numba** : il accélère les boucles numériques, mais compiler du calcul qui ne pèse presque rien n'aurait rien changé au temps total. Le levier qui paie, c'est de **paralléliser les téléchargements** (`ThreadPoolExecutor`) plutôt que de les enchaîner un par un : pendant qu'un thread attend le réseau, les autres travaillent. Les threads suffisent, parce que l'attente réseau libère le GIL de Python – c'est précisément le cas d'usage où ils sont efficaces. On complète par de la vectorisation NumPy sur le calcul de la composition en acides aminés.

Mesures

Comparaison des deux endpoints via l'API, pour les tailles de lot demandées :

Taille du lot	/ingest (s)	/ingest_fast (s)	Gain
1 élément	0,164	0,077	53,2 %
100 éléments	6,869	2,093	69,5 %

L'objectif de 30 % est donc largement dépassé, et l'écart se creuse avec la taille du lot (le parallélisme ne paie qu'en volume). En complément, nous avons mesuré le même levier sur l'ingestion complète des 418 protéines : 53,8 s en séquentiel contre 11,6 s en parallèle, soit un gain de 78,5 %.

7. Guide d'utilisation

Prérequis

Python 3.12, Docker (pour MinIO) et uv pour la gestion des dépendances.

Installation

```
uv venv --python 3.12
source .venv/bin/activate           # Windows : .venv\Scripts\Activate.ps1
uv pip install -e ".[all]"
cp .env.example .env                # valeurs par défaut : MinIO local
```

Démarrer le stockage

```
docker compose up -d minio minio-setup # MinIO + creation des 3 buckets
python scripts/check_minio.py           # verifie la connexion et les buckets
```

La console web de MinIO est accessible sur <http://localhost:9001> (identifiants minioadmin / minioadmin).

Exécuter le pipeline

Le plus simple est de laisser DVC rejouer les quatre étapes dans l'ordre ; il ne recalcule que ce dont les entrées ont changé :

```
dvc repro           # ingest -> features -> curate -> train
```

Chaque étape peut aussi être lancée séparément :

```
python scripts/ingest.py --limit 20      # ingestion (--limit pour un test rapide)
python scripts/build_features.py         # raw -> staging (features)
python scripts/build_curated.py          # staging -> curated (DuckDB)
python scripts/train.py                  # entraînement du modele -> metrics.json
```

Lancer l'API

```
python -m uvicorn oncolake.api.main:app --reload
```

La documentation interactive est alors disponible sur <http://localhost:8000/docs>. Les endpoints `/raw`, `/staging` et `/curated` donnent un aperçu de chaque zone ; `/health` et `/stats` renseignent sur l'état du lac.

Exemple d'appel à l'ingestion (le même corps fonctionne pour `/ingest_fast`) :

```
POST http://localhost:8000/ingest
{
  "items": [
    {"accession": "P04637", "sequence": "MEEPQSDPSV..."},
    {"accession": "P01116", "sequence": "MTEYKLVVVG..."}
  ]
}
```

Reproduire les mesures de performance

L'API doit tourner dans un autre terminal, puis :

```
python scripts/bench_api.py      # compare /ingest et /ingest_fast sur des lots de
1 et 100
```

Le script écrit un rapport comparatif dans `logs/endpoint_benchmark.md`.

8. Conclusion

Nous avons mis en place un data lake complet et reproductible : deux sources de données différentes, une architecture en trois zones, un pipeline que l'on peut relancer entièrement d'une seule commande, et une API pour consulter les données. Le modèle ne parvient pas à distinguer les deux familles de protéines à partir de leur forme ; nous assumons ce résultat et l'expliquons plutôt que de le masquer.

Pour aller plus loin, il faudrait sans doute des informations plus fines sur les protéines, ou une question un peu différente. Ce projet nous aura surtout appris qu'un travail sur les données demande autant d'attention aux cas particuliers et à la reproductibilité qu'au modèle lui-même.